1.12 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-1.1 Write a short Python function, is_multiple(n, m), that takes two integer values and returns True if n is a multiple of m, that is, $n = mi$ for some integer i, and False otherwise.

R-1.2 Write a short Python function, is_even(k), that takes an integer value and returns True if k is even, and **False** otherwise. However, your function cannot use the multiplication, modulo, or division operators.

R-1.3 Write a short Python function, minmax(data), that takes a sequence of one or more numbers, and returns the smallest and largest numbers, in the form of a tuple of length two. Do not use the built-in functions min or max in implementing your solution.

R-1.4 Write a short Python function that takes a positive integer n and returns the sum of the squares of all the positive integers smaller than n.

R-1.5 Give a single command that computes the sum from Exercise R-1.4, relying on Python's comprehension syntax and the built-in sum function.

R-1.6 Write a short Python function that takes a positive integer n and returns the sum of the squares of all the odd positive integers smaller than n.

R-1.7 Give a single command that computes the sum from Exercise R-1.6, relying on Python's comprehension syntax and the built-in sum function.

R-1.8 Python allows negative integers to be used as indices into a sequence, such as a string. If string s has length n, and expression s[k] is used for index $-n \leq k < 0$, what is the equivalent index $j \geq 0$ such that s[j] references the same element?

R-1.9 What parameters should be sent to the range constructor, to produce a range with values 50, 60, 70, 80?

R-1.10 What parameters should be sent to the range constructor, to produce a range with values 8, 6, 4, 2, 0, -2, -4, -6, -8?

R-1.11 Demonstrate how to use Python's list comprehension syntax to produce the list [1, 2, 4, 8, 16, 32, 64, 128, 256].

R-1.12 Python's random module includes a function choice(data) that returns a random element from a non-empty sequence. The random module includes a more basic function randrange, with parameterization similar to the built-in range function, that return a random choice from the given range. Using only the randrange function, implement your own version of the choice function.

Creativity

C-1.13 Write a pseudo-code description of a function that reverses a list of n integers, so that the numbers are listed in the opposite order than they were before, and compare this method to an equivalent Python function for doing the same thing.

C-1.14 Write a short Python function that takes a sequence of integer values and determines if there is a distinct pair of numbers in the sequence whose product is odd.

C-1.15 Write a Python function that takes a sequence of numbers and determines if all the numbers are different from each other (that is, they are distinct).

C-1.16 In our implementation of the scale function (page 25), the body of the loop executes the command data[j] *= factor. We have discussed that numeric types are immutable, and that use of the *= operator in this context causes the creation of a new instance (not the mutation of an existing instance). How is it still possible, then, that our implementation of scale changes the actual parameter sent by the caller?

C-1.17 Had we implemented the scale function (page 25) as follows, does it work properly?

```
def scale(data, factor):
  for val in data:
    val *= factor
```

Explain why or why not.

C-1.18 Demonstrate how to use Python's list comprehension syntax to produce the list [0, 2, 6, 12, 20, 30, 42, 56, 72, 90].

C-1.19 Demonstrate how to use Python's list comprehension syntax to produce the list ['a', 'b', 'c', ..., 'z'], but without having to type all 26 such characters literally.

C-1.20 Python's random module includes a function shuffle(data) that accepts a list of elements and randomly reorders the elements so that each possible order occurs with equal probability. The random module includes a more basic function randint(a, b) that returns a uniformly random integer from a to b (including both endpoints). Using only the randint function, implement your own version of the shuffle function.

C-1.21 Write a Python program that repeatedly reads lines from standard input until an EOFError is raised, and then outputs those lines in reverse order (a user can indicate end of input by typing ctrl-D).

C-1.22 Write a short Python program that takes two arrays a and b of length n storing **int** values, and returns the dot product of a and b. That is, it returns an array c of length n such that $c[i] = a[i] \cdot b[i]$, for $i = 0, \ldots, n-1$.

C-1.23 Give an example of a Python code fragment that attempts to write an element to a list based on an index that may be out of bounds. If that index is out of bounds, the program should catch the exception that results, and print the following error message:
"`Don't try buffer overflow attacks in Python!`"

C-1.24 Write a short Python function that counts the number of vowels in a given character string.

C-1.25 Write a short Python function that takes a string s, representing a sentence, and returns a copy of the string with all punctuation removed. For example, if given the string `"Let's try, Mike."`, this function would return `"Lets try Mike"`.

C-1.26 Write a short program that takes as input three integers, a, b, and c, from the console and determines if they can be used in a correct arithmetic formula (in the given order), like "$a+b=c$," "$a=b-c$," or "$a*b=c$."

C-1.27 In Section 1.8, we provided three different implementations of a generator that computes factors of a given integer. The third of those implementations, from page 41, was the most efficient, but we noted that it did not yield the factors in increasing order. Modify the generator so that it reports factors in increasing order, while maintaining its general performance advantages.

C-1.28 The ***p-norm*** of a vector $v = (v_1, v_2, \ldots, v_n)$ in n-dimensional space is defined as

$$\|v\| = \sqrt[p]{v_1^p + v_2^p + \cdots + v_n^p}.$$

For the special case of $p = 2$, this results in the traditional ***Euclidean norm***, which represents the length of the vector. For example, the Euclidean norm of a two-dimensional vector with coordinates $(4,3)$ has a Euclidean norm of $\sqrt{4^2+3^2} = \sqrt{16+9} = \sqrt{25} = 5$. Give an implementation of a function named `norm` such that `norm(v, p)` returns the p-norm value of v and `norm(v)` returns the Euclidean norm of v. You may assume that v is a list of numbers.

Projects

P-1.29 Write a Python program that outputs all possible strings formed by using the characters `'c'`, `'a'`, `'t'`, `'d'`, `'o'`, and `'g'` exactly once.

P-1.30 Write a Python program that can take a positive integer greater than 2 as input and write out the number of times one must repeatedly divide this number by 2 before getting a value less than 2.

P-1.31 Write a Python program that can "make change." Your program should take two numbers as input, one that is a monetary amount charged and the other that is a monetary amount given. It should then return the number of each kind of bill and coin to give back as change for the difference between the amount given and the amount charged. The values assigned to the bills and coins can be based on the monetary system of any current or former government. Try to design your program so that it returns as few bills and coins as possible.

P-1.32 Write a Python program that can simulate a simple calculator, using the console as the exclusive input and output device. That is, each input to the calculator, be it a number, like `12.34` or `1034`, or an operator, like `+` or `=`, can be done on a separate line. After each such input, you should output to the Python console what would be displayed on your calculator.

P-1.33 Write a Python program that simulates a handheld calculator. Your program should process input from the Python console representing buttons that are "pushed," and then output the contents of the screen after each operation is performed. Minimally, your calculator should be able to process the basic arithmetic operations and a reset/clear operation.

P-1.34 A common punishment for school children is to write out a sentence multiple times. Write a Python stand-alone program that will write out the following sentence one hundred times: "I will never spam my friends again." Your program should number each of the sentences and it should make eight different random-looking typos.

P-1.35 The ***birthday paradox*** says that the probability that two people in a room will have the same birthday is more than half, provided n, the number of people in the room, is more than 23. This property is not really a paradox, but many people find it surprising. Design a Python program that can test this paradox by a series of experiments on randomly generated birthdays, which test this paradox for $n = 5, 10, 15, 20, \ldots, 100$.

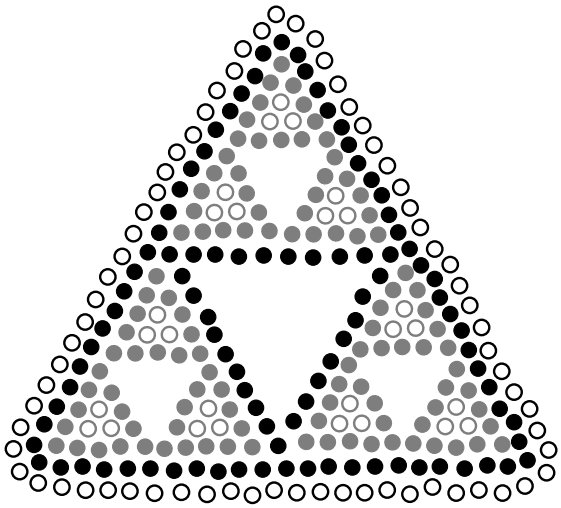
P-1.36 Write a Python program that inputs a list of words, separated by whitespace, and outputs how many times each word appears in the list. You need not worry about efficiency at this point, however, as this topic is something that will be addressed later in this book.

Chapter Notes

The official Python Web site (http://www.python.org) has a wealth of information, including a tutorial and full documentation of the built-in functions, classes, and standard modules. The Python interpreter is itself a useful reference, as the interactive command help(foo) provides documentation for any function, class, or module that foo identifies.

Books providing an introduction to programming in Python include titles authored by Campbell *et al.* [22], Cedar [25], Dawson [32], Goldwasser and Letscher [43], Lutz [72], Perkovic [82], and Zelle [105]. More complete reference books on Python include titles by Beazley [12], and Summerfield [91].

Chapter 4 Recursion



Contents

4.1 Illustrative Examples 150
4.1.1 The Factorial Function 150
4.1.2 Drawing an English Ruler 152
4.1.3 Binary Search 155
4.1.4 File Systems 157
4.2 Analyzing Recursive Algorithms 161
4.3 Recursion Run Amok 165
4.3.1 Maximum Recursive Depth in Python 168
4.4 Further Examples of Recursion 169
4.4.1 Linear Recursion 169
4.4.2 Binary Recursion 174
4.4.3 Multiple Recursion 175
4.5 Designing Recursive Algorithms 177
4.6 Eliminating Tail Recursion 178
4.7 Exercises 180

One way to describe repetition within a computer program is the use of loops, such as Python's while-loop and for-loop constructs described in Section 1.4.2. An entirely different way to achieve repetition is through a process known as ***recursion***.

Recursion is a technique by which a function makes one or more calls to itself during execution, or by which a data structure relies upon smaller instances of the very same type of structure in its representation. There are many examples of recursion in art and nature. For example, fractal patterns are naturally recursive. A physical example of recursion used in art is in the Russian Matryoshka dolls. Each doll is either made of solid wood, or is hollow and contains another Matryoshka doll inside it.

In computing, recursion provides an elegant and powerful alternative for performing repetitive tasks. In fact, a few programming languages (e.g., Scheme, Smalltalk) do not explicitly support looping constructs and instead rely directly on recursion to express repetition. Most modern programming languages support functional recursion using the identical mechanism that is used to support traditional forms of function calls. When one invocation of the function make a recursive call, that invocation is suspended until the recursive call completes.

Recursion is an important technique in the study of data structures and algorithms. We will use it prominently in several later chapters of this book (most notably, Chapters 8 and 12). In this chapter, we begin with the following four illustrative examples of the use of recursion, providing a Python implementation for each.

- The ***factorial function*** (commonly denoted as $n!$) is a classic mathematical function that has a natural recursive definition.
- An ***English ruler*** has a recursive pattern that is a simple example of a fractal structure.
- ***Binary search*** is among the most important computer algorithms. It allows us to efficiently locate a desired value in a data set with upwards of billions of entries.
- The ***file system*** for a computer has a recursive structure in which directories can be nested arbitrarily deeply within other directories. Recursive algorithms are widely used to explore and manage these file systems.

We then describe how to perform a formal analysis of the running time of a recursive algorithm and we discuss some potential pitfalls when defining recursions. In the balance of the chapter, we provide many more examples of recursive algorithms, organized to highlight some common forms of design.